

Product Blueprint: Conversational Knowledge Management System

1. Project Overview & Value Proposition

A multi-tenant, web-based knowledge management platform that structures note-taking like chat applications (e.g., WhatsApp). Instead of organizing notes by human contacts, users organize their stream of consciousness into a rigid **two-tier hierarchy (Topics → Subtopics)**. Notes are stored sequentially as a linear, time-ordered feed of rich-text bubbles, optimizing rapid information ingestion and effortless retrieval.

2. Technical Stack Expansion

Here is your original stack augmented with necessary frameworks, libraries, and AWS services (marked as **[ADDED]**) to make it a fully functional production system:

Frontend (UI/UX)

- **Core:** React.js, TypeScript **[ADDED for type safety across API calls]**
- **Styling & Components:** Tailwind CSS & shadcn/ui **[ADDED for rapid, beautiful UI prototyping]**
- **Rich Text Editor:** React-Markdown or Quill.js **[ADDED for chat-bubble formatting]**
- **State Management / API Fetching:** TanStack Query (React Query) **[ADDED for easy caching and chat feed syncing]**
- **IDE:** Visual Studio Code

Backend & Security

- **Core Engine:** Spring Boot (Java), Spring Web
- **Security & Multi-Tenancy:** Spring Security + Spring Security OAuth2 Client **[ADDED for Google/GitHub login and JWT generation]**
- **API Testing:** Postman
- **IDE:** IntelliJ IDEA

Data & Infrastructure

- **Database:** MongoDB (using Spring Data MongoDB for seamless object mapping)
- **Containerization:** Docker
- **Orchestration:** Kubernetes (K8s)
- **Cloud Provider (AWS Ecosystem [ADDED]):**
 - **EKS (Elastic Kubernetes Service):** To host your K8s cluster.
 - **DocumentDB:** AWS's fully managed, MongoDB-compatible database engine (safer than self-hosting Mongo inside K8s for a tight timeline).
 - **ECR (Elastic Container Registry):** To host your private Docker images.

- **Logging & Observability:** Kibana (part of the EFK/ELK stack deployed inside your cluster)
 - **Domain Management:** Squarespace / Google Domains + AWS Route 53 **[ADDED to route domain traffic to AWS]**
 - **Design & Architecture:** Draw.io
-

3. Core Database Schema Design (MongoDB)

To support multi-tenancy, every document will anchor to a `userId`.

```
// "topics" Collection
{
  "_id": "ObjectId",
  "userId": "String (OAuth Subject ID)",
  "name": "String (e.g., Technology)",
  "createdAt": "ISODate"
}

// "subtopics" Collection
{
  "_id": "ObjectId",
  "topicId": "ObjectId (References topics)",
  "userId": "String",
  "name": "String (e.g., Java)",
  "createdAt": "ISODate"
}

// "notes" Collection
{
  "_id": "ObjectId",
  "subtopicId": "ObjectId (References subtopics)",
  "userId": "String",
  "content": "String (Markdown Text)",
  "isEdited": "Boolean",
  "isArchived": "Boolean",
  "createdAt": "ISODate",
  "updatedAt": "ISODate"
}
```

4. 30-Day Aggressive Sprint Schedule (Backend-First)

Given your timeline, we split this into strict weekly milestones.



Week 1: Core Backend & Database Setup

- **Goal:** Build a robust, secured local API engine.
- **Tasks:**
 - Initialize Spring Boot project and configure Spring Data MongoDB.
 - Implement **Spring Security OAuth2** for GitHub & Google login. Generate a **JWT token** upon successful login to pass to the client.
 - Develop CRUD REST APIs for Topics, Subtopics, and Notes (Messages).
 - Implement **Soft Delete** and **Archive** toggles on the Notes API.
 - Build the **Global Search API** using MongoDB Text Indexes to search across topic titles, subtopic titles, and note contents simultaneously.
 - *Milestone:* All APIs thoroughly tested and validated via **Postman**.



Week 2: Frontend Development & API Integration

- **Goal:** Create a responsive, intuitive WhatsApp-style UI.
- **Tasks:**
 - Scaffold React app with Tailwind CSS and layout a 3-column UI: Sidebar (Topics  Subtopics) | Chat Window (Notes Feed) | Global Search Overlays.
 - Integrate Google/GitHub OAuth sign-in flow.
 - Wire up React endpoints to Spring Boot APIs using TanStack Query.
 - Integrate a Markdown editor/renderer into the message input field to support rich text.
 - *Milestone:* Application is fully functional on your local machine (localhost).



Week 3: Containerization & AWS Infrastructure Setup

- **Goal:** Package the apps and spin up the AWS ecosystem.
- **Tasks:**
 - Write optimized Dockerfiles for both React (using Nginx) and Spring Boot.
 - Spin up AWS EKS (Kubernetes) and AWS DocumentDB (MongoDB).
 - Create private repositories in AWS ECR and push your Docker images manually.
 - Write Kubernetes manifests (deployment.yaml, service.yaml, ingress.yaml) for backend, frontend, and database configurations.
 - *Milestone:* Containers successfully communicating inside the local Docker environment and initial deployment ready for AWS.



Week 4: Cloud Deployment, Logging & Polish

- **Goal:** Go live, secure traffic, and ensure system observability.
- **Tasks:**
 - Apply manifests to AWS EKS cluster.
 - Deploy **Elasticsearch/Fluentd/Kibana (EFK)** or standard Logstash cluster inside K8s to aggregate Spring Boot application logs.
 - Configure an AWS ALB (Application Load Balancer) or Ingress Controller.

- Update Squarespace DNS records to point to the AWS Load Balancer using **AWS Route 53** and attach an SSL certificate via AWS Certificate Manager (ACM) for HTTPS.
- Final end-to-end testing, bug squashing, and performance verification.
- *Milestone:* Live production app accessible securely via your personal domain.



Product Manager's Strategic Recommendations

1. **Scope Creep Warning:** Do not add features like "image/file attachments," "voice notes," or "message reactions" during this month. Stick strictly to Markdown text, archiving, and global search.
2. **AWS Costs:** Kubernetes (EKS) and DocumentDB on AWS carry fixed baseline costs (\$70+ per month minimum for EKS management fees alone). Since this is a personal app, keep your node groups small (e.g., using t3.medium instances) or leverage AWS Fargate to pay only for exact pod usage to save money.

Your plan is incredibly solid and highly educational! To help you get moving instantly, would you like me to **draft the Dockerfiles for your React and Spring Boot services**, or should we **write out the MongoDB Java Entity structures** to kickstart Week 1?